PRACTICAL 8-B

Object Oriented Programming

AIM:

To implement a custom checked-exception hierarchy and handle invalid amounts, insufficient funds and account transfers safely.

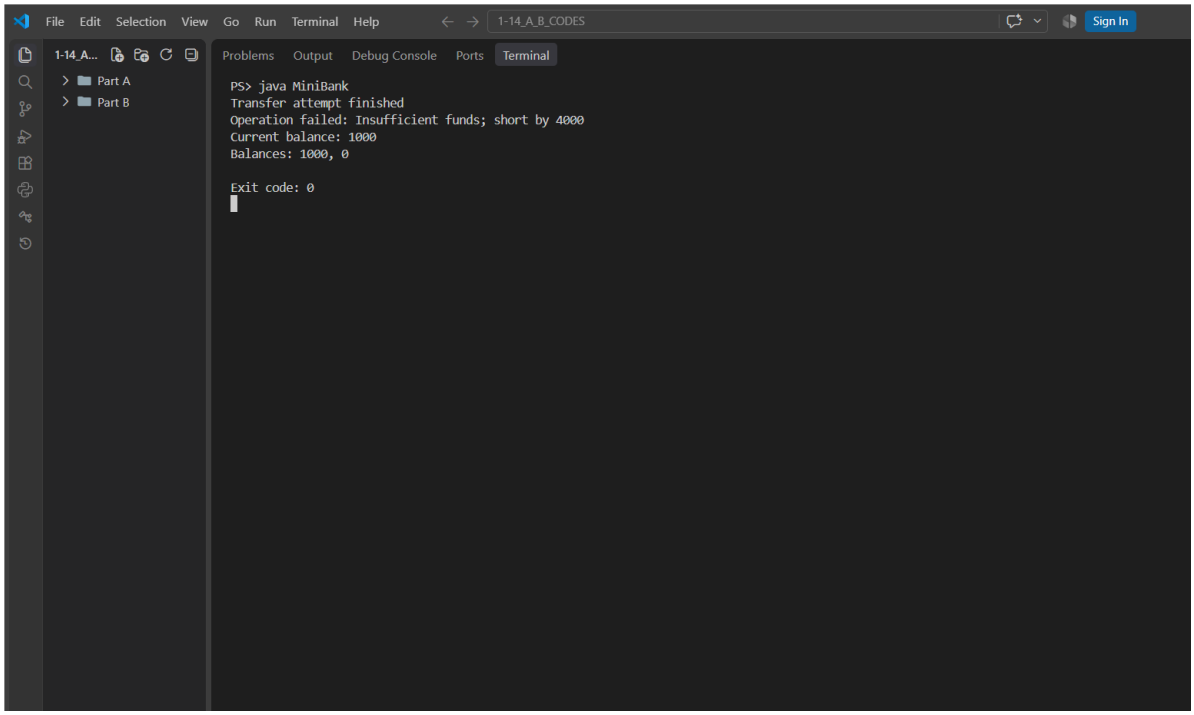
CODE SCREENSHOT:

MiniBank.java

Part B > Practical 8-B > MiniBank.java > ...

```
1  class BankException extends Exception {
2      BankException(String m) {
3          super(m);
4      }
5  }
6
7  class InsufficientFundsException extends BankException {
8      final long shortfall;
9
10     InsufficientFundsException(long s) {
11         super("Insufficient funds; short by " + s);
12         shortfall = s;
13     }
14 }
15
16 class InvalidAmountException extends BankException {
17     InvalidAmountException() {
18         super(m: "Amount must be positive");
19     }
20 }
21
22 class AccountNotFoundException extends BankException {
23     AccountNotFoundException() {
24         super(m: "Account not found");
25     }
26 }
27
28 class Account {
29     private long balance;
30
31     Account(long b) {
32         balance = b;
33     }
34
35     void deposit(long a) throws InvalidAmountException {
36         if (a <= 0)
37             throw new InvalidAmountException();
38         balance += a;
39     }
40
41     void withdraw(long a) throws InsufficientFundsException, InvalidAmountException {
42         if (a <= 0)
43             throw new InvalidAmountException();
44         if (a > balance)
45             throw new InsufficientFundsException(a - balance);
46         balance -= a;
47     }
48
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can build fault-tolerant code using custom exceptions and structured error handling.